Processingをはじめよう

第8章

動き　その１

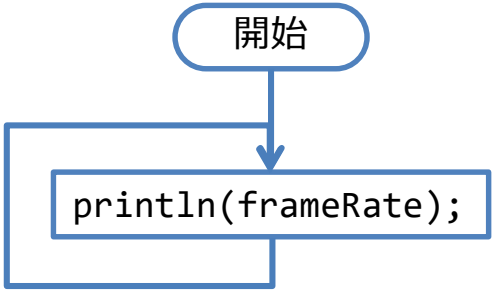
目次

- フレーム
- スピードと方向
- 2点間の移動
- 乱数
- タイマー
- 円運動
- Robot 6: Motion

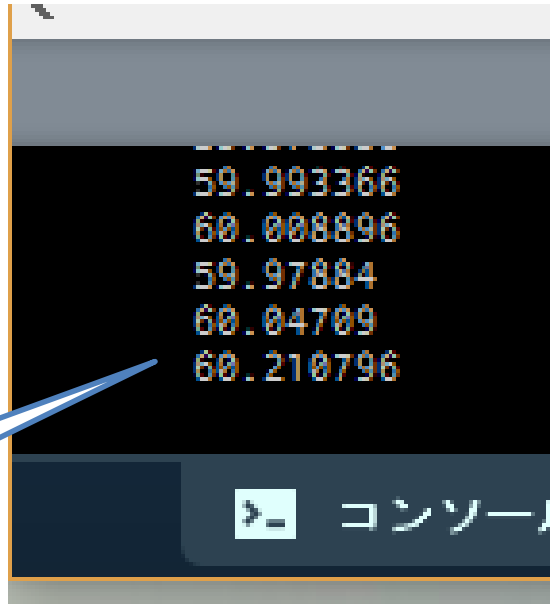
フレーム

- Example 8-1 フレームレートを見る

```
void draw() {
  println(frameRate);
}
```



- frameRate変数
  - 教科書 P225
  - 1秒間に何回画面が更新するか
  - 通常 1秒で60フレーム



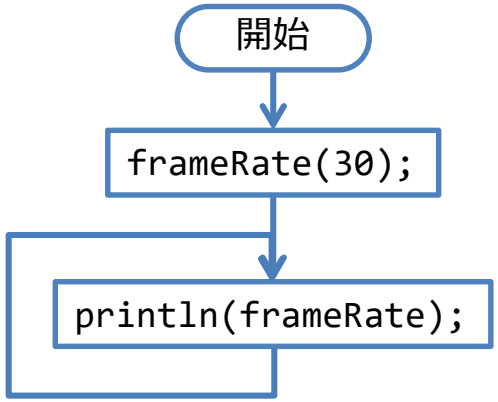
コンソールに
フレームレート表示

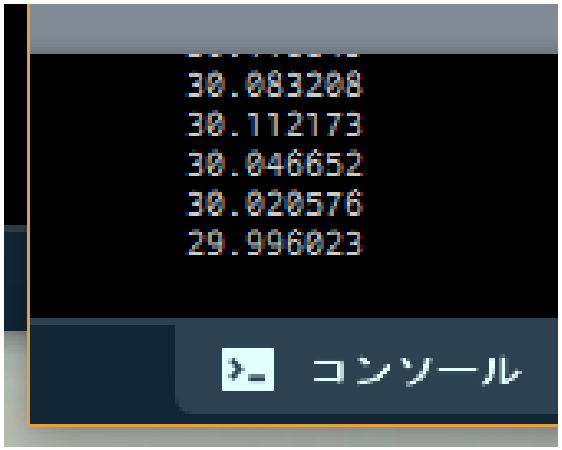
フレーム

- Example 8-2 フレームレートを見る

```
void setup() {
  frameRate(30);          //毎秒30フレームに設定
  //frameRate(12);        //毎秒12フレームに設定
  //frameRate(2);         //毎秒2フレームに設定
  //frameRate(0.5);       //毎秒0.5フレームに設定
}

void draw() {
  println(frameRate);
}
```

設定を変更して
フレームレートを確認





- frameRate()関数
  - 教科書 P225
  - フレームレートを設定

スピードと方向

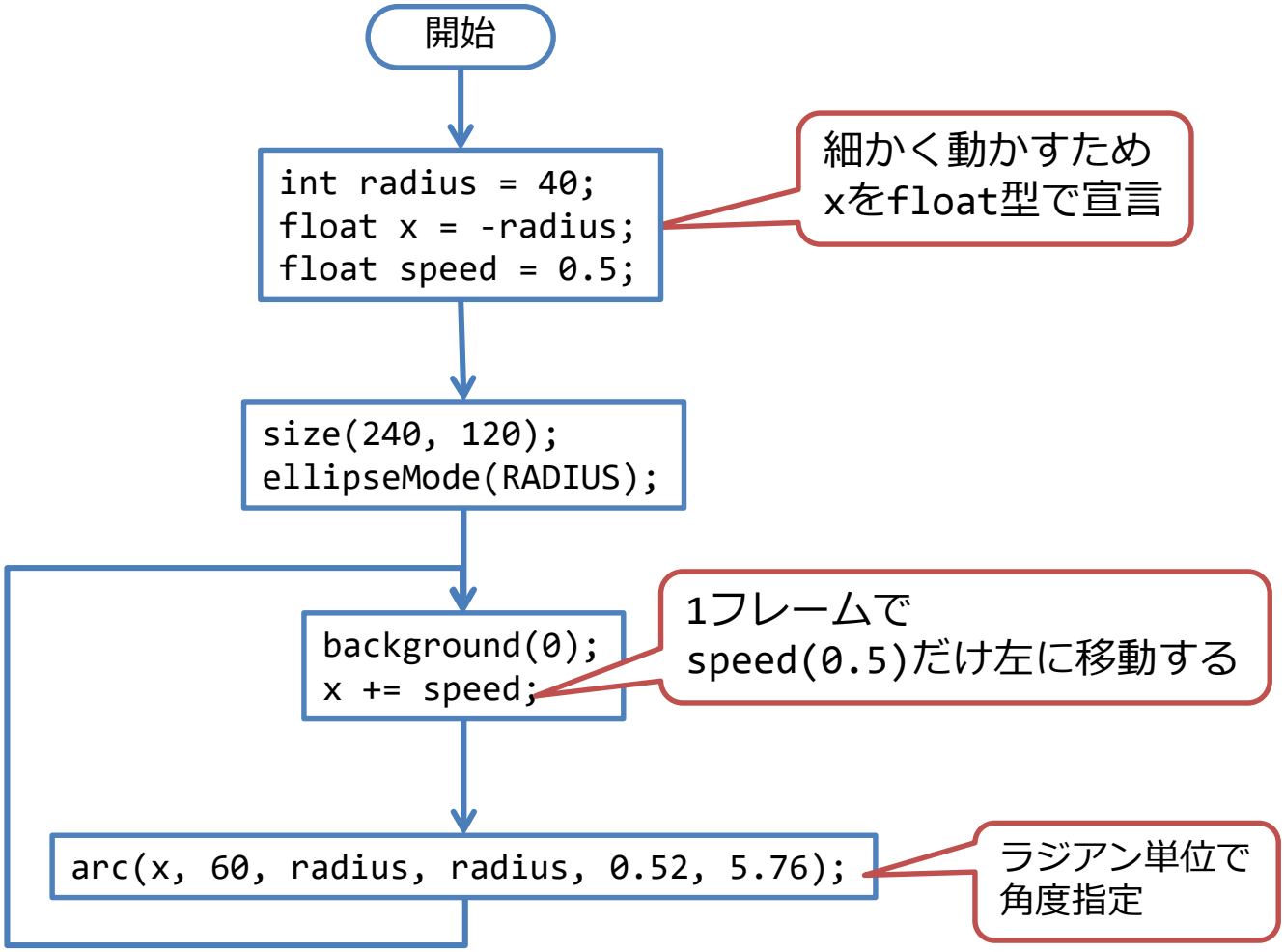
- Example 8-3 図形の移動

```
int radius = 40;          // 図形の半径radius       初期値40
float x = -radius;        // 図形のx座標            初期値 -radius
float speed = 0.5;        // 図形の移動する値speed  初期値0.5

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);    // 円を半径を使って指定する
}

void draw() {
  background(0);
  x += speed;             // xをspeed加算する(xを左にspeedだけ移動)
  arc(x, 60, radius, radius, 0.52, 5.76); //円弧を描く
}
```

- 角度を指定して円弧を書く
  - arc(x, y, w, h, start, stop)　start：開始角度, stop：終了角度

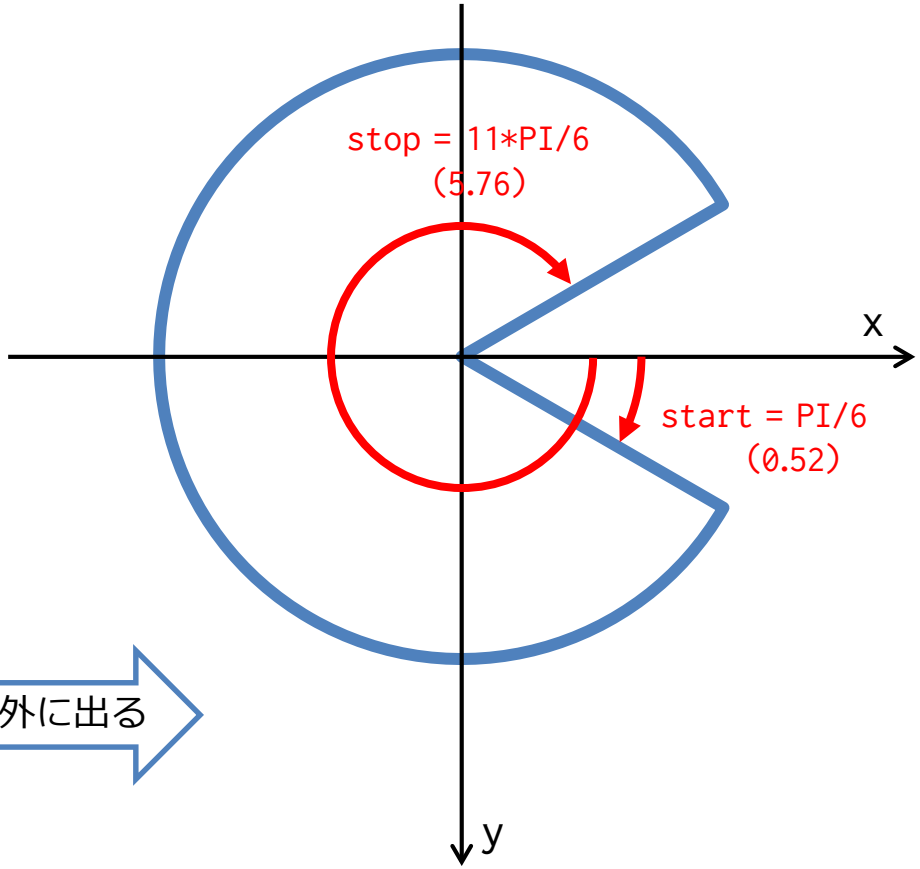
スピードと方向

- フローチャート



実行画面　開始・終了角度

開始角度 start　30度（PI/6）
終了角度 stop　330度（11*PI/6）



スピードと方向

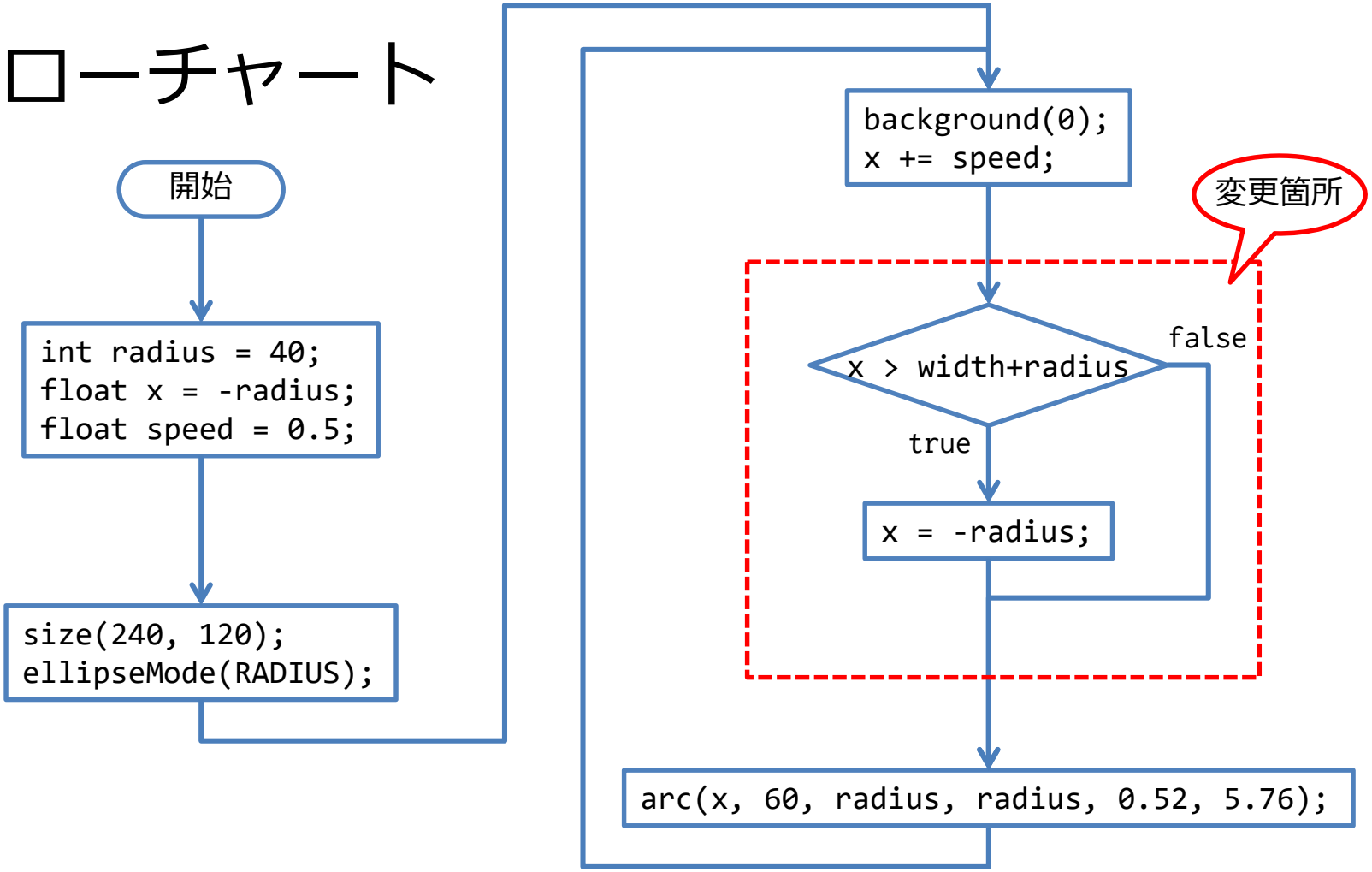
- Example 8-4 円筒形を回す

```
int radius = 40;
float x = -radius;
float speed = 0.5;

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
}

void draw() {
  background(0);
  x += speed;                    // xをspeedだけ加算
  if (x > width+radius) {        // 右の画面の外に出たら
    x = -radius;                 // 左から戻ってくる
  }
  arc(x, 60, radius, radius, 0.52, 5.76);
}
```

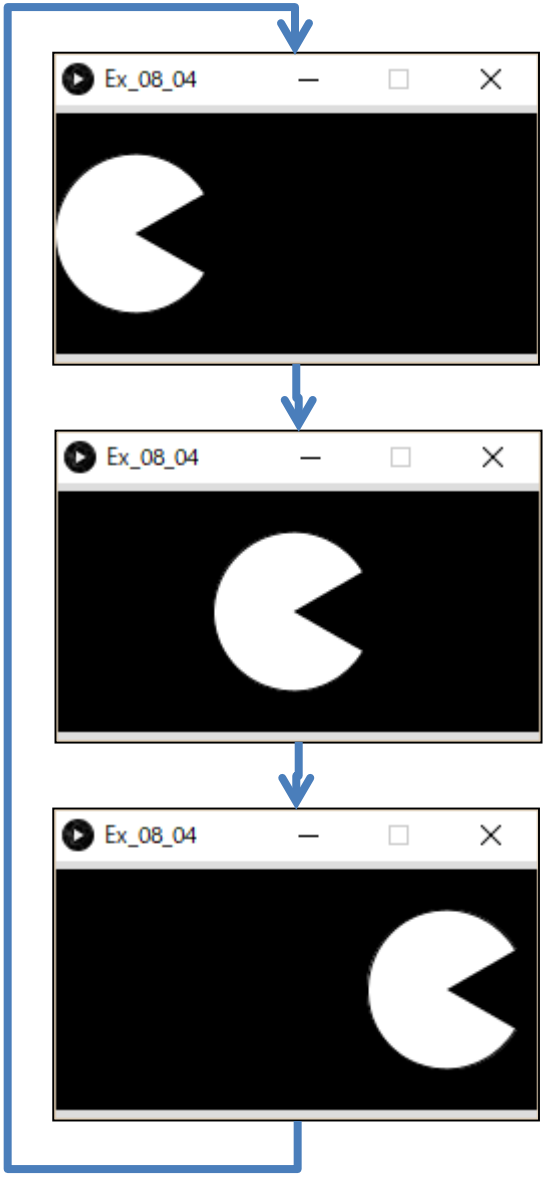
変更箇所

スピードと方向

- フローチャート



実行画面



- 図形の初期位置は
  左端外側（x = -radius）
- 図形が左から右へ移動
  （x += speed）
- 右側から外へ出たら
  左側から入ってくる

```
if(x > width+radius){
  x = -radius;
}
```

スピードと方向

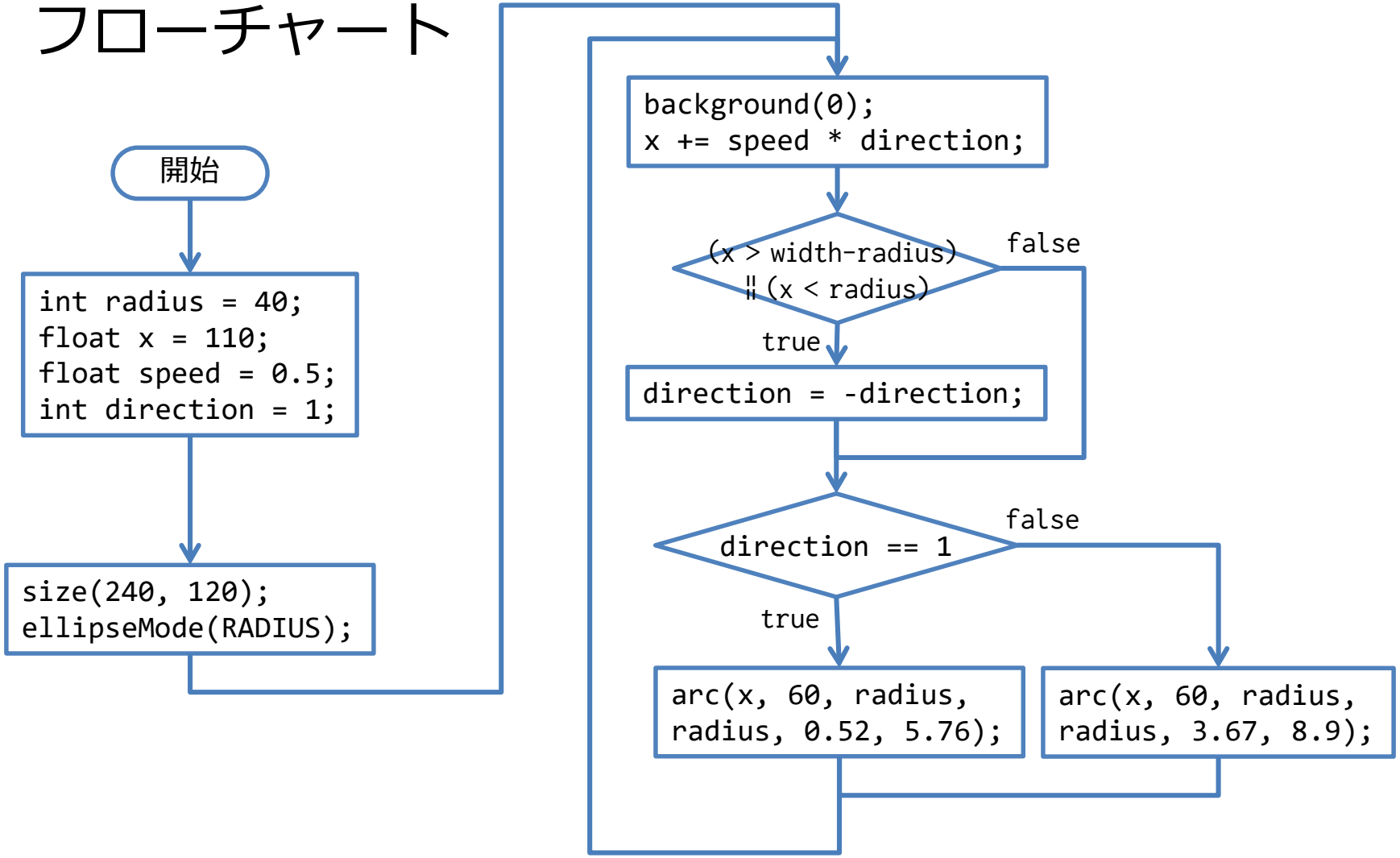
- Example 8-5 壁に当たって跳ね返る

```
int radius = 40;   // 図形の半径
float x = 110;     // x座標の初期値110
float speed = 0.5; // 1フレームで移動する量
int direction = 1; // 移動する方向(1:右、-1:←)

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
}
```

```
void draw() {
  background(0);
  x += speed * direction; //(移動量)×(方向)加算
  if ((x > width-radius) || (x < radius)) { //衝突したら
    direction = -direction; // 方向を反転
  }
  if (direction == 1) { // 進行方向が右の場合
    arc(x, 60, radius, radius, 0.52, 5.76); //口が右向き
  } else { //それ以外
    arc(x, 60, radius, radius, 3.67, 8.9);  //口が左向き
  }
}
```

スピードと方向

- フローチャート



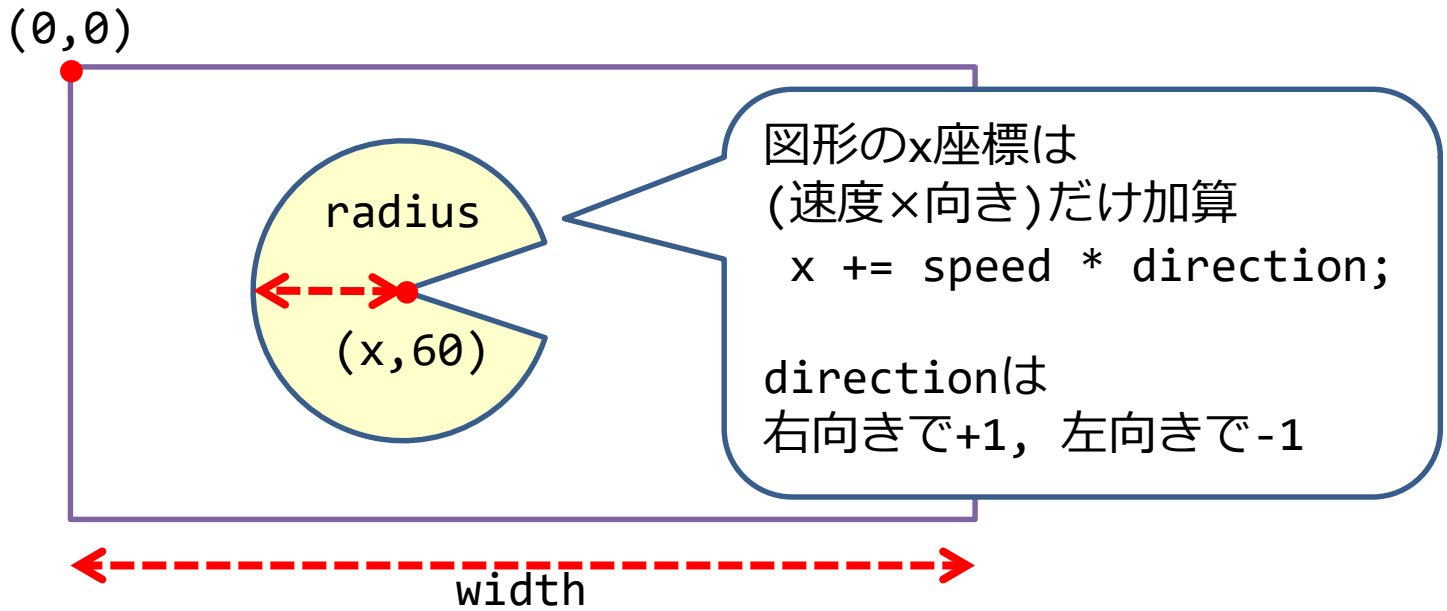
スピードと方向

- ポイント
  - 左端・右端で図形を反射させる
  - 移動量を(speed * direction)に変更
    - directionの符号を変えると移動方向を変更できる
  - 反射する条件：2つの条件のどちらかが真の場合
    - 右端： x > width-radius
    - 左端： x < radius
  - 条件が真のとき、移動方向を反転する
    - direction = -direction;

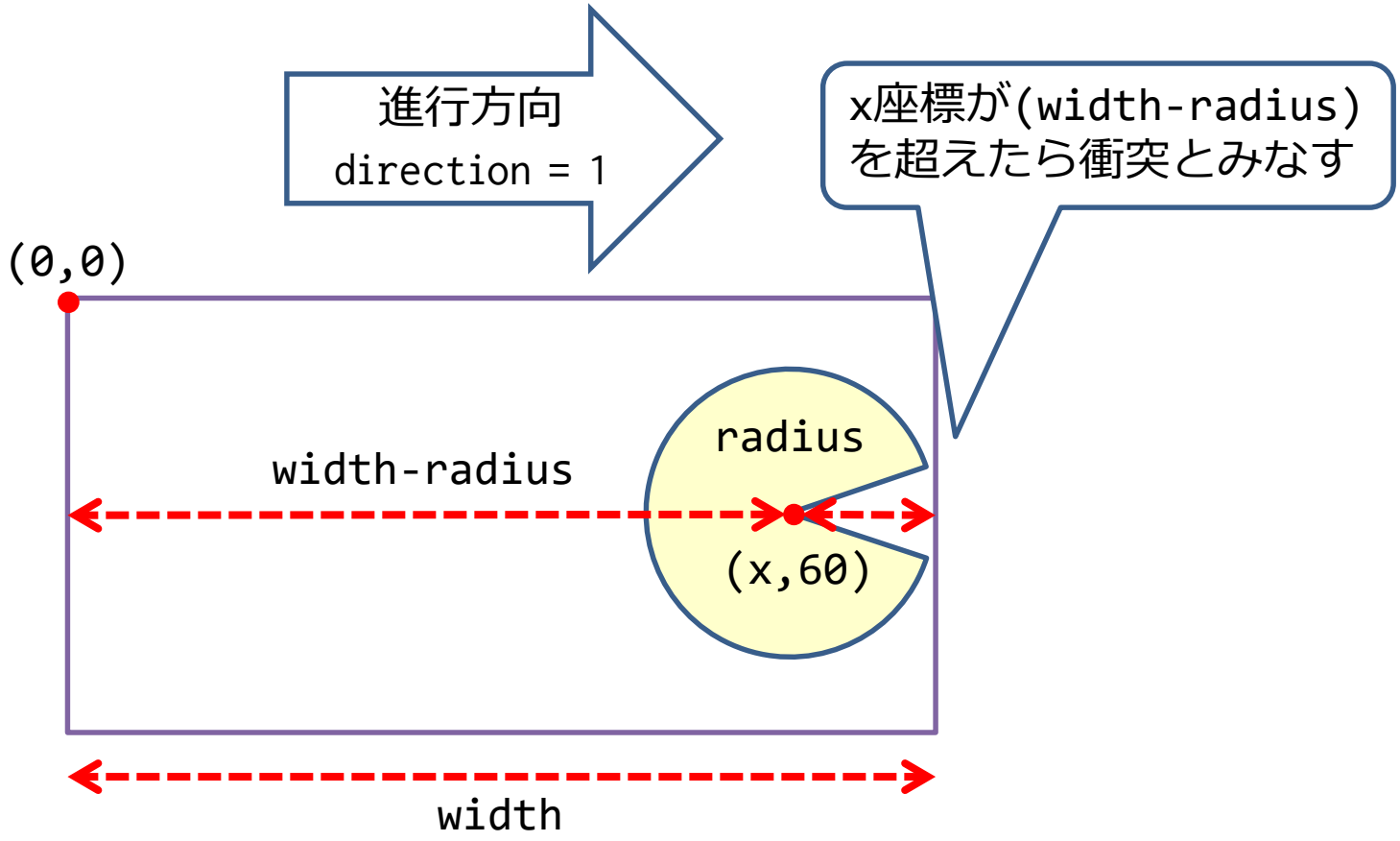
符号を変えて方向を反転

実行結果

Example_7_5
Example_7_5
Example_7_5
右へ移動して…
右端で反射
左へ移動して…
左端で反射…
を繰り返す

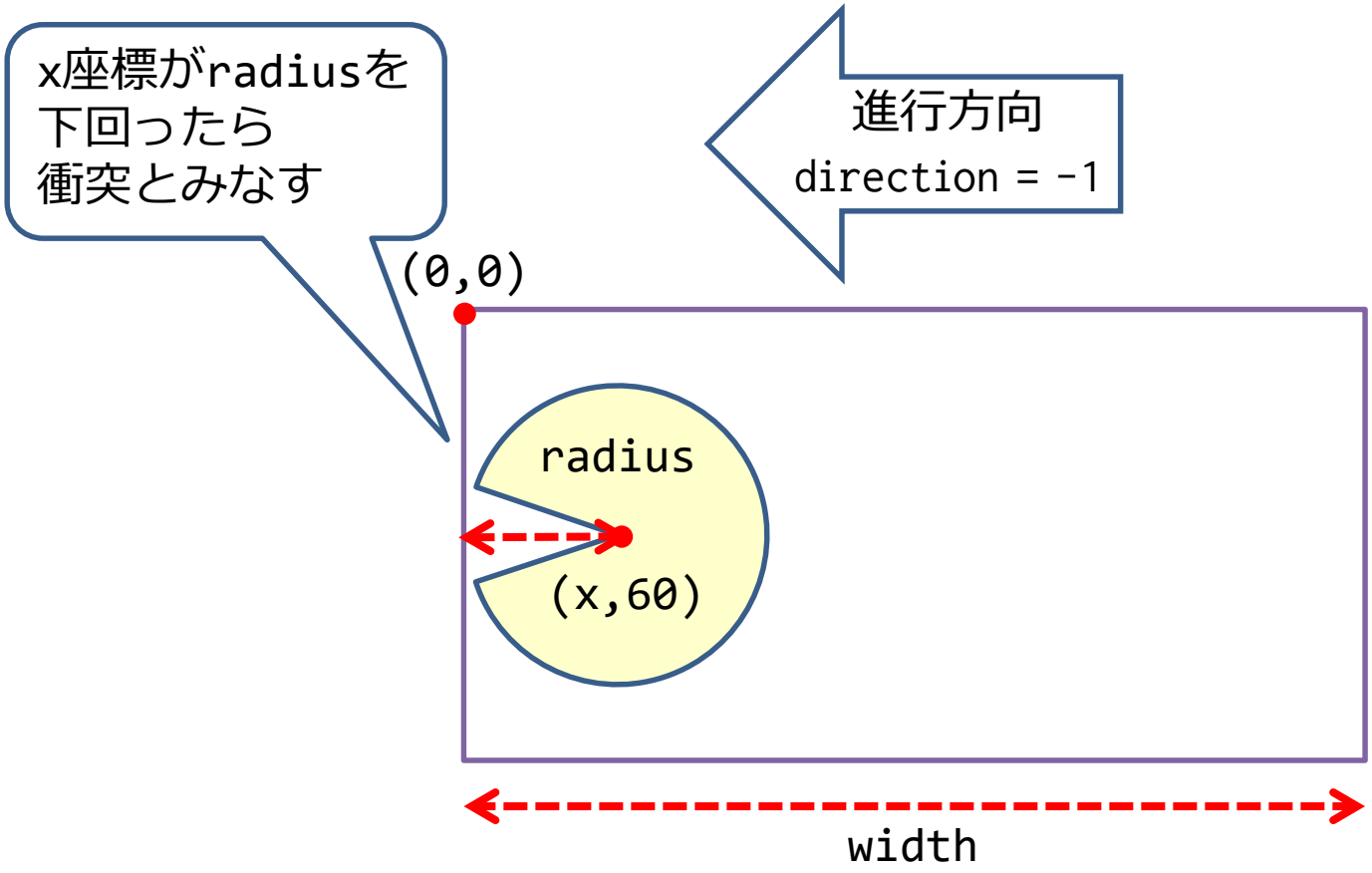

(0,0)
radius
(x,60)
width
図形のx座標は
(速度×向き)だけ加算
x += speed * direction;
directionは
右向きで+1，左向きで-1

右端で反射する場合




```
if(x > width - radius) {    // 右壁に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

左端で反射する場合




```
if(x < radius) {              // 左壁に衝突したら
  direction = -direction;     // 進行方向を逆にする
}
```

左端・右端で反射する条件をまとめる

```
if(x > width - radius) {    // 右端に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

```
if(x < radius) {            // 左端に衝突したら
  direction = -direction;   // 進行方向を逆にする
}
```

論理和 ||
上の2つの条件の
どちらか一方を満たす

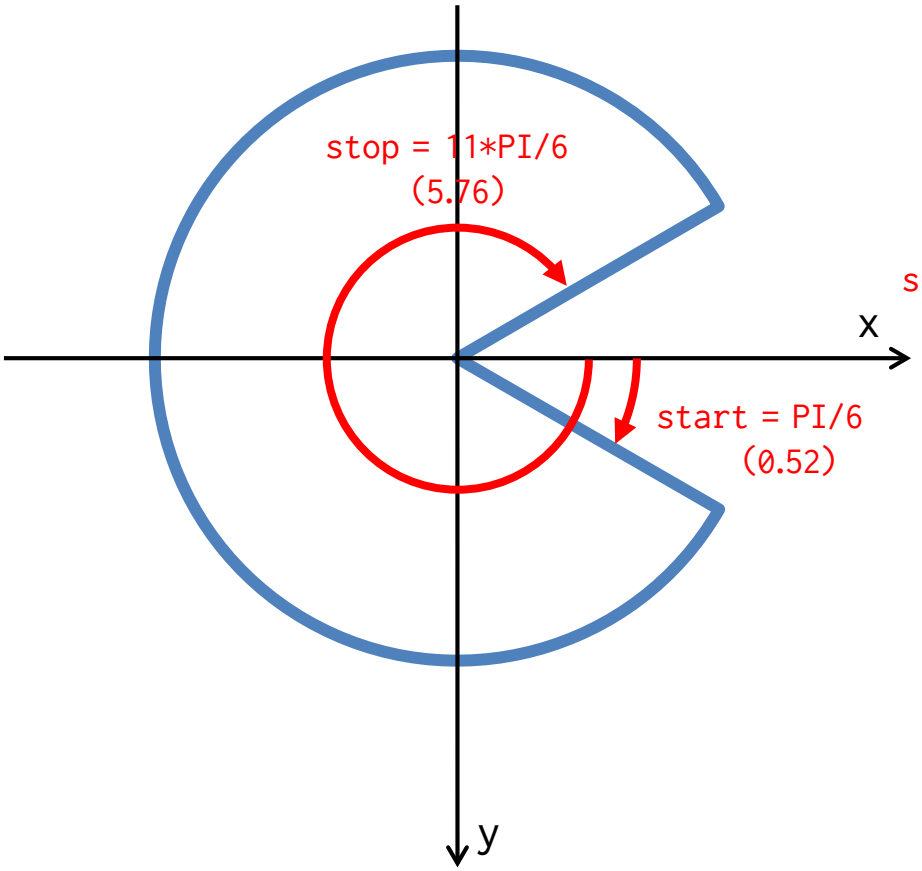
```
if((x > width - radius) || (x < radius)) { // 右端または左端に衝突したら
  direction = -direction; // 進行方向を逆にする
}
```

論理和・論理積

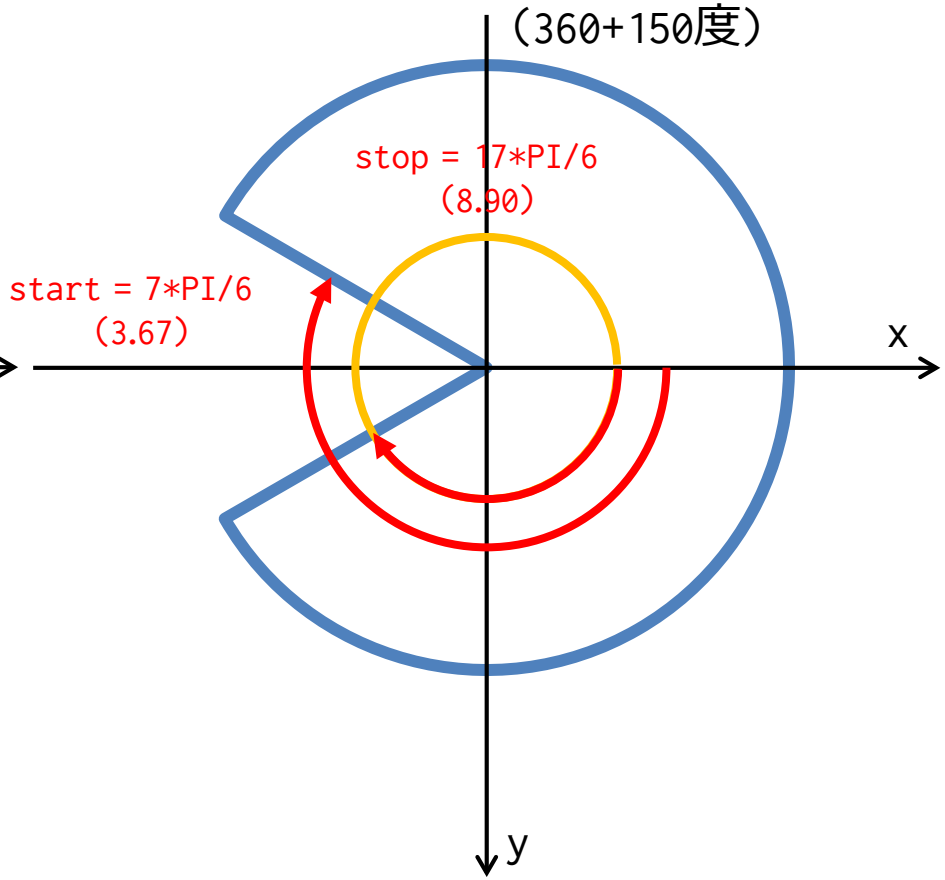
- 論理和　||
  - （条件1）||（条件2）
    条件1 または 条件2が成り立つ場合
    （どちらかの条件が真の場合）
    例）`if (mousePressed || keyPressed)`
- 論理積　&&
  - （条件1）&&（条件2）
    条件1かつ条件2が成り立つ場合(どちらも真の場合)
    例）`if (mousePressed && mouseButton == LEFT)`

開始・終了角度

右向き
開始角度 start　30度（PI/6）
終了角度 stop　330度（11*PI/6）

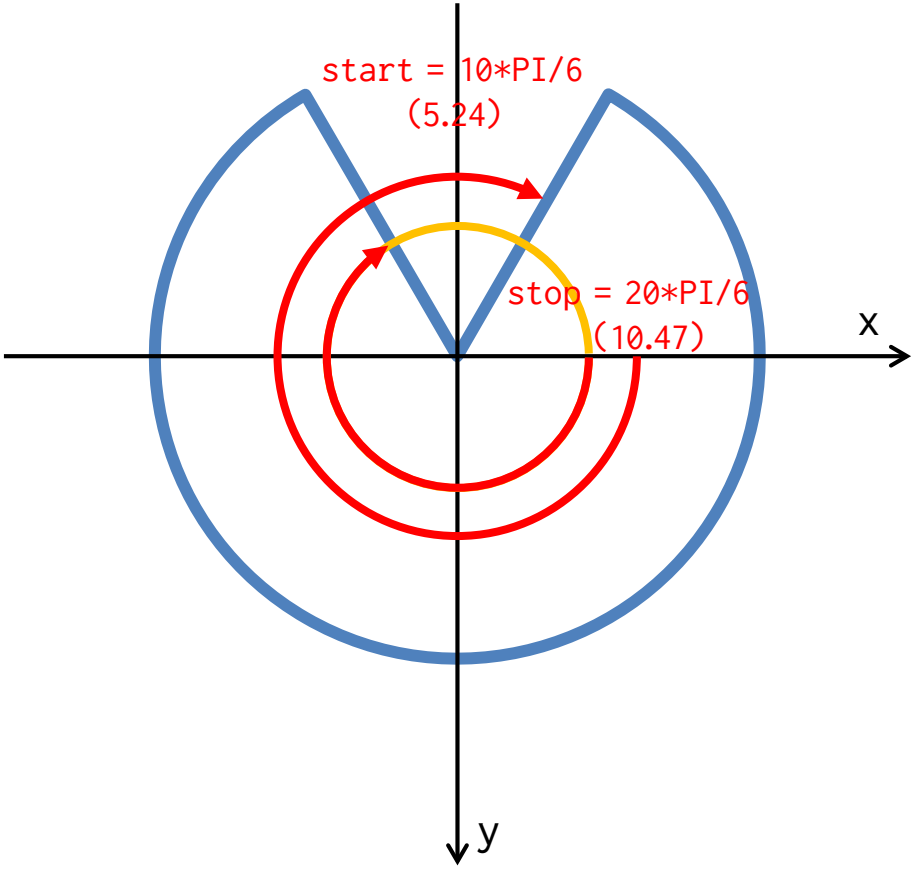
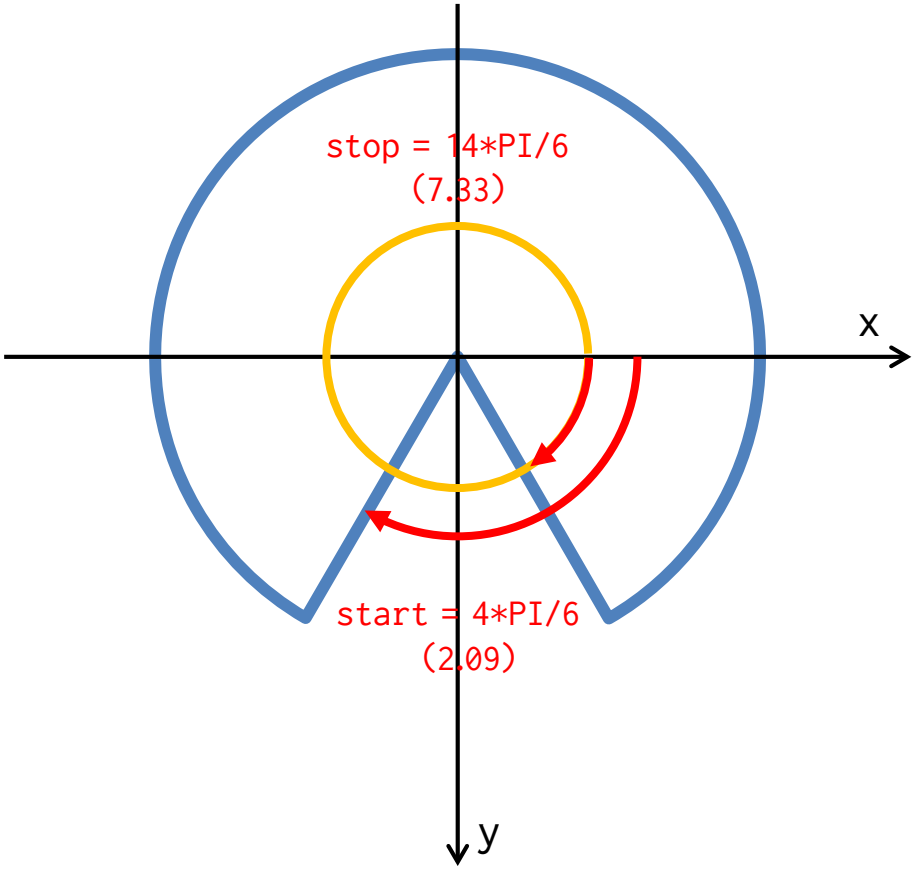


左向き
開始角度 start 210度（7*PI/6）
終了角度 stop　510度（17*PI/6）
（360+150度）



おまけ　上向き・下向き

上向き
開始角度 start 300度（10*PI/6）
終了角度 stop　600度（20*PI/6）



下向き
開始角度 start 120度（4*PI/6）
終了角度 stop　420度（14*PI/6）